

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA

JULIANA RODRIGUES

**Relatório de Análise Experimental:
Algoritmos de Emparelhamento em Grafos
Bipartidos**

Porto Alegre

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

Reitora: Prof^a. Marcia Cristina Bernardes Barbosa

Vice-Reitor: Prof. Pedro de Almeida Costa

Pró-Reitora de Pós-Graduação: Prof^a. Claudia Wasserman

Diretor do Instituto de Informática: Prof. Luciano Paschoal Gaspar

:

Bibliotecário-chefe do Instituto de Informática: Alexsander Borges Ribeiro

1 RELATÓRIO DO EXPERIMENTO

1.1 Introdução

Este relatório descreve a avaliação empírica de quatro algoritmos de emparelhamento máximo ponderado em grafos bipartidos: o **Algoritmo de Caminhos Aumentantes Simples** (SimpleAugmentingMatcher), o **Algoritmo de Hopcroft-Karp** (HopcroftKarpMatcher), e duas variantes do **Algoritmo Húngaro** (HungarianMatcher) que se diferenciam pela estratégia de busca de caminho mais curto: Bellman-Ford (BellmanFordStrategy) e Johnson/Dijkstra (JohnsonDijkstraStrategy).

O objetivo é verificar as complexidades assintóticas teóricas por meio de medições empíricas, identificar os pontos de cruzamento de desempenho entre os algoritmos, e analisar o impacto da densidade do grafo (α) sobre o comportamento prático. A análise é organizada em torno de três eixos: (i) nível do laço principal (contagem de fases/aumentações), (ii) custo do laço interno (arestas visitadas por busca), e (iii) tempo de parede total.

O problema resolvido é o *emparelhamento máximo de peso máximo* em grafos bipartidos: encontrar um emparelhamento de cardinalidade máxima que, entre todos os de mesma cardinalidade, maximiza o peso total das arestas. Para grafos sem peso, o critério reduz-se à máxima cardinalidade.

1.2 Instâncias de Teste e Plano de Experimentos

1.2.1 Estrutura dos Grafos Gerados

Os grafos de teste são grafos bipartidos esparsos com $2n$ vértices (n em cada partição S e T) e aproximadamente $m = n^\alpha$ arestas, para $1 \leq \alpha \leq 2$. Os pesos são inteiros uniformes em $[-n^2, n^2]$, sorteados aleatoriamente. O parâmetro n é o número de vértices por lado da bipartição.

A estrutura de dados central é a classe `BipartiteGraph`, que armazena o grafo como uma **matriz de pesos** e uma **matriz booleana de adjacência**, ambas de tamanho $n \times n$:

Listing 1.1 – Estrutura de dados `BipartiteGraph` (`BipartiteGraph.h`)

```
1 class BipartiteGraph {
```

```

2 private:
3     int n;
4     std::vector<std::vector<int>> weight_matrix;
5     std::vector<std::vector<bool>> edge_exists;
6 public:
7     static constexpr int INF_NEGATIVE = std::numeric_limits<int>::min();
8     explicit BipartiteGraph(int vertices_per_partition);
9     int get_n() const;
10    void set_edge(int u, int v, int weight);
11    bool has_edge(int u, int v) const;
12    int get_weight(int u, int v) const;
13 };

```

Essa representação permite acesso a peso e existência de aresta em $O(1)$, ao custo de $O(n^2)$ de memória. Para os experimentos com $\alpha < 2$, o grafo é esparso e a matriz tem muitos zeros; para $\alpha = 2$ o grafo é denso e a representação matricial é plenamente justificada.

1.2.2 Plano de Experimentos

Os experimentos estão organizados em três grupos (Tabela ??), estratificados ao longo de dois eixos ortogonais: o *nível de observação algorítmica* (laço principal, busca interna, tempo de parede) e o *parâmetro de grafo variado* (n , α ou faixa de peso):

Tabela 1.1 – Resumo dos experimentos realizados.

Teste	Grupo	Objetivo	n variado	Reps.
U0	Correção	Oráculo não ponderado	2–6	50
W0	Correção	Oráculo ponderado	2–4	100
U1	Laço principal	Iterações vs. n	50–5000	10
U2	Busca interna	Arestas BFS vs. defeito	200–2000	5
U3	Escalabilidade	Tempo de parede vs. n	100–10000	20
U4	Sensibilidade	Tempo vs. α (n fixo)	500–2000	15
W1	Laço principal	Déficit BF vs. JD	50–1000	10
W2	Custo por chamada	Relaxações por chamada	50–2000	10
W3	Escalabilidade	Tempo ponderado vs. n	50–2000	20
W4	Estresse	Regimes de peso negativos	100–1000	20
C1	Comparação	Tabela final não ponderada	100–10000	30
C2	Comparação	Tabela final ponderada	50–2000	20

O expoente de densidade α toma os valores $\{1,0; 1,25; 1,5; 1,75; 2,0\}$ nos testes de escalabilidade ($U3$, $W3$, $C1$, $C2$) e $\{1,0; 1,5; 2,0\}$ nos demais. O orçamento de repetições é assimétrico: testes exploratórios usam 10 repetições, escalabilidade usa 20 e os testes de relatório final usam 30, refletindo a necessidade de intervalos de confiança mais estreitos para o ajuste de curvas.

1.2.3 Estrutura dos Arquivos CSV

Cada invocação do binário de *benchmark* gera um arquivo no diretório *raw/* e, após a conclusão de todas as repetições, o script de orquestração concatena esses arquivos em *data/combined/{test_id}.csv*. O esquema universal de colunas é:

```
test_id, n, alpha, rep, algorithm, iteration, weight_regime,
metric_name, metric_value
```

A coluna *iteration* vale -1 para métricas escalares por execução (como *wall_clock_ms*) e o índice do passo para séries temporais (como *defect_at_iteration*). Isso distingue claramente observações de nível de execução das séries de convergência internas.

1.2.4 Ambiente de Execução

Todos os experimentos foram executados na máquina descrita na Tabela ??.

Tabela 1.2 – Configuração do ambiente de execução.

Componente	Especificação
Processador	Intel Core Ultra 5 236V, 8 núcleos, 1 thread/núcleo, até 4,7 GHz
Cache L1	320 KiB dados + 512 KiB instruções (8 instâncias)
Cache L2	14 MiB (5 instâncias)
Cache L3	8 MiB (1 instância)
Memória RAM	16 GiB
Sistema operacional	Ubuntu 24.04.4 LTS (Noble Numbat), kernel 6.17.0-35-generic
Compilador	g++ 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
Flags de compilação	-std=c++17 -O2

Os processos concorrentes do sistema operacional não foram controlados expli-

tamente; os tempos reportados correspondem à mediana de múltiplas repetições independentes, o que atenua o impacto de interferências pontuais.

1.3 Algoritmos e Análise de Complexidade

1.3.1 Algoritmo de Caminhos Aumentantes Simples

O `SimpleAugmentingMatcher` implementa o algoritmo clássico de Ford para emparelhamento máximo em grafos bipartidos: para cada vértice $u \in S$, tenta encontrar um caminho aumentante por meio de uma busca em profundidade (DFS) que alterna entre arestas fora e dentro do emparelhamento atual.

Listing 1.2 – Núcleo da DFS em `SimpleAugmentingMatcher.cpp`

```

1 bool SimpleAugmentingMatcher::dfs(int u, const BipartiteGraph& g,
2   std::vector<bool>& visited, std::vector<int>& match_T) {
3   int n = g.get_n();
4   for (int v = 0; v < n; ++v) {
5       if (g.has_edge(u, v) && !visited[v]) {
6           visited[v] = true;
7           if (match_T[v] < 0 || dfs(match_T[v], g, visited, match_T)) {
8               match_T[v] = u;
9               return true;
10          }
11      }
12  }
13  return false;
14 }
```

Complexidade: O laço externo percorre n vértices de S . Para cada um, a DFS pode visitar $O(n)$ vértices e $O(m)$ arestas no pior caso. A complexidade total é $O(n \cdot m) = O(n^{1+\alpha})$.

O vetor `visited` é reinicializado a cada chamada do laço externo, garantindo que a busca não visite um vértice de T duas vezes na mesma tentativa de aumento.

1.3.2 Algoritmo de Hopcroft-Karp

O `HopcroftKarpMatcher` melhora o algoritmo simples agrupando em *fases*: em cada fase, uma BFS constrói uma rede em camadas (*layered network*) sobre o grafo de emparelhamento atual, e uma DFS encontra um conjunto máximo de caminhos aumentantes

de comprimento mínimo e *vertex-disjoint*.

Listing 1.3 – Fase BFS de construção da rede em camadas (HopcroftKarpMatcher.cpp)

```

1 bool HopcroftKarpMatcher::bfs(const BipartiteGraph& g,
2     const std::vector<int>& match_S, const std::vector<int>& match_T,
3     std::vector<int>& dist) {
4     std::queue<int> q;
5     for (int u = 0; u < n; ++u) {
6         if (match_S[u] < 0) { dist[u] = 0; q.push(u); }
7         else { dist[u] = INF; }
8     }
9     dist[n] = INF;
10    while (!q.empty()) {
11        int u = q.front(); q.pop();
12        if (dist[u] < dist[n]) {
13            for (int v = 0; v < n; ++v) {
14                if (g.has_edge(u, v)) {
15                    if (match_T[v] < 0) dist[n] = dist[u] + 1;
16                    else if (dist[match_T[v]] == INF) {
17                        dist[match_T[v]] = dist[u] + 1;
18                        q.push(match_T[v]);
19                    }
20                }
21            }
22        }
23    }
24    return dist[n] != INF;
25 }

```

A DFS subsequente usa o array `dist` para manter-se nas camadas e define `dist[u] = INF` quando esgota os vizinhos de `u`, evitando revisitas na mesma fase:

Listing 1.4 – Fase DFS de aumento em camadas (HopcroftKarpMatcher.cpp)

```

1 bool HopcroftKarpMatcher::dfs(int u, const BipartiteGraph& g,
2     std::vector<int>& match_S, std::vector<int>& match_T,
3     std::vector<int>& dist) {
4     if (u != n) {
5         for (int v = 0; v < n; ++v) {
6             if (g.has_edge(u, v) &&
7                 (match_T[v] < 0 || dist[match_T[v]] == dist[u] + 1)) {
8                 if (match_T[v] < 0 || dfs(match_T[v], g, match_S, match_T, dist)) {
9                     match_T[v] = u; match_S[u] = v;
10                    return true;
11                }
12            }
13        }
14        dist[u] = INF;
15        return false;
16    }
17    return true;
18 }

```

Complexidade: Hopcroft e Karp provaram que são necessárias no máximo $O(\sqrt{n})$ fases. Cada fase executa uma BFS e uma DFS em $O(m)$, logo a complexidade total é $O(\sqrt{n} \cdot m) = O(n^{0,5+\alpha})$.

1.3.3 Algoritmo Húngaro com Bellman-Ford

O HungarianMatcher resolve o emparelhamento de peso máximo por meio de aumentações sucessivas ao longo do caminho mais curto (em custo negado) na rede residual. A variante Bellman-Ford (BellmanFordStrategy) suporta pesos negativos sem modificação.

Listing 1.5 – Laço de relaxação de Bellman-Ford (BellmanFordStrategy.cpp)

```

1 for (int step = 0; step < 2 * n && relaxed; ++step) {
2     relaxed = false;
3     for (int u = 0; u < n; ++u) {
4         for (int v = 0; v < n; ++v) {
5             if (g.has_edge(u, v)) {
6                 // Aresta forward S->T (fora do emparelhamento)
7                 if (match_S[u] != v &&
8                     distances[u] != INF && distances[u] - weight < distances[n+v]) {
9                     distances[n+v] = distances[u] - weight;
10                    predecessors[n+v] = u; relaxed = true;
11                }
12                // Aresta reverse T->S (dentro do emparelhamento)
13                if (match_T[v] == u &&
14                    distances[n+v] != INF && distances[n+v] + weight < distances[u]) {
15                    distances[u] = distances[n+v] + weight;
16                    predecessors[u] = n+v; relaxed = true;
17                }
18            }
19        }
20    }
21 }

```

O código usa o truque de negação de peso ($-weight$ nas arestas forward) para converter o problema de caminho mais pesado em caminho mais curto. A terminação antecipada ($\&\& relaxed$) reduz o custo em instâncias práticas.

Complexidade: Bellman-Ford em um grafo de $2n$ vértices e m arestas executa em $O(n \cdot m)$ por chamada. Com n aumentações no total, a complexidade é $O(n^2 \cdot m) = O(n^{2+\alpha})$.

1.3.4 Algoritmo Húngaro com Johnson/Dijkstra

A variante `JohnsonDijkstraStrategy` substitui Bellman-Ford por Dijkstra com *potenciais de Johnson*, que transformam os pesos negativos em não-negativos após cada aumento, mantendo a invariância $d'_{uv} \geq 0$ para todas as arestas na rede residual.

Listing 1.6 – Inicialização dos potenciais e transformação de pesos (`JohnsonDijkstraStrategy.cpp`)

```

1 void JohnsonDijkstraStrategy::initialize_potentials(const BipartiteGraph& g) {
2     int W = max_edge_weight(g);
3     for (int v = 0; v < n; ++v)
4         potentials[n + v] = -W; // potencial inicial para todos os vertices de T
5 }
6
7 int JohnsonDijkstraStrategy::get_transformed_weight(
8     const BipartiteGraph& g, int u, int v, bool is_forward) const {
9     if (is_forward) {
10         int orig_cost = -g.get_weight(u, v); // negado para maximizar peso
11         return orig_cost - (potentials[n + v] - potentials[u]);
12     } else {
13         int orig_cost = g.get_weight(u, v);
14         return orig_cost - (potentials[u] - potentials[n + v]);
15     }
16 }

```

Após cada aumento, os potenciais são atualizados com as distâncias calculadas pelo Dijkstra:

Listing 1.7 – Atualização dos potenciais após cada Dijkstra (`JohnsonDijkstraStrategy.cpp`)

```

1 for (int i = 0; i < n * 2; ++i) {
2     if (distances[i] <= delta)
3         potentials[i] += distances[i];
4     else
5         potentials[i] += delta;
6 }

```

Essa atualização garante que na próxima chamada todos os custos transformados sejam não-negativos, permitindo o uso de Dijkstra com fila de prioridade (min-heap).

Complexidade: Dijkstra com heap binário em grafo de m arestas executa em $O(m \log n)$ por chamada. Com n aumentações: $O(n \cdot m \log n) = O(n^{1+\alpha} \log n)$.

1.3.5 Interface de Estratégia e Desacoplamento

Os dois algoritmos Húngaros são desacoplados via interface polimórfica `IPathfindingStrategy`, que define o contrato de busca do caminho aumentante mais curto:

Listing 1.8 – Interface `IPathfindingStrategy` (`IPathfindingStrategy.h`)

```

1 class IPathfindingStrategy {
2 public:
3     static constexpr int INF = 1e9;
4     virtual bool find_shortest_augmenting_path(
5         const BipartiteGraph& g,
6         const std::vector<int>& match_S,
7         const std::vector<int>& match_T,
8         std::vector<int>& predecessors,
9         std::vector<int>& distances) = 0;
10 };

```

O `HungarianMatcher` recebe um `shared_ptr<IPathfindingStrategy>` no construtor e delega toda a busca de caminho à estratégia injetada, sem conhecimento de qual variante está em uso.

1.3.6 Resumo Teórico

A Tabela ?? sintetiza as complexidades teóricas dos quatro algoritmos em função de n e α .

Tabela 1.3 – Complexidades teóricas dos algoritmos ($m = n^\alpha$).

Algoritmo	Complexidade	Expoente em n
SimpleAugmentingMatcher	$O(n \cdot m) = O(n^{1+\alpha})$	$1 + \alpha$
HopcroftKarpMatcher	$O(\sqrt{n} \cdot m) = O(n^{0,5+\alpha})$	$0,5 + \alpha$
HungarianMatcher + BF	$O(n^2 \cdot m) = O(n^{2+\alpha})$	$2 + \alpha$
HungarianMatcher + JD	$O(n \cdot m \log n) = O(n^{1+\alpha} \log n)$	$1 + \alpha (+\log)$

1.4 Resultados Experimentais e Análise

1.4.1 Verificação de Correção (U0 e W0)

Antes de qualquer análise de desempenho, os algoritmos foram validados contra oráculos de força bruta em instâncias pequenas.

U0 (não ponderado): Para $n \in \{2, 3, 4, 5, 6\}$ e $\alpha \in \{1,0; 1,5; 2,0\}$ com 50 repetições por par, ambos SimpleAugmentingMatcher e HopcroftKarpMatcher retornaram `matching_size == oracle_size` em **100% das instâncias**. O oráculo enumera todos os subconjuntos de arestas e seleciona o de maior cardinalidade sem vértice repetido.

W0 (ponderado): Para $n \in \{2, 3, 4\}$, $\alpha = 1,5$ e 100 repetições, ambas as variantes Húngaras produziram `matching_size == oracle_size` e $|\text{matching_value} - \text{oracle_value}| < 10^{-6}$ em **100% das instâncias**, incluindo casos com pesos completamente negativos. Os valores do oráculo foram confirmados como *máximos*, não mínimos — verificando a convenção de maximização do peso.

Os testes de correção atuam como portões: somente após ambos passarem é que as análises de desempenho subsequentes procedem.

1.4.2 U1 – Iterações do Laço Principal vs. n

O teste U1 mede o número de iterações do laço externo (aumentações para Simple, fases para HK) em função de n ; os expoentes ajustados estão na Tabela ??.

Tabela 1.4 – Expoentes empíricos do laço principal (U1, ajuste k em n^k).

α	Algoritmo	$k_{\text{teórico}}$	k_{emp}	R^2
1,0	SimpleAugmentingMatcher	1,0	1,01	1,00
1,0	HopcroftKarpMatcher	0,5	0,17	0,88
1,5	SimpleAugmentingMatcher	1,0	1,00	1,00
1,5	HopcroftKarpMatcher	0,5	0,00	1,00
2,0	SimpleAugmentingMatcher	1,0	1,00	1,00
2,0	HopcroftKarpMatcher	0,5	0,00	1,00

O Simple executa exatamente n aumentações em todas as densidades ($k_{\text{emp}} \approx 1,0$), confirmando a implementação do laço externo. O HK, porém, revela um resultado

surpreendente: para $\alpha = 1,5$ e $\alpha = 2,0$, o número de fases é **constante** ($k_{\text{emp}} = 0$), tipicamente 1 a 2 fases para qualquer n testado. Isso indica que, em grafos suficientemente densos, HK encontra o emparelhamento perfeito em uma única rodada BFS+DFS — confirmando que os grafos gerados possuem estrutura favorável ao emparelhamento. Para $\alpha = 1,0$ (esparso), as fases crescem levemente com $k_{\text{emp}} = 0,17$, ainda bem abaixo do pior caso $O(\sqrt{n})$ (Figura ??).

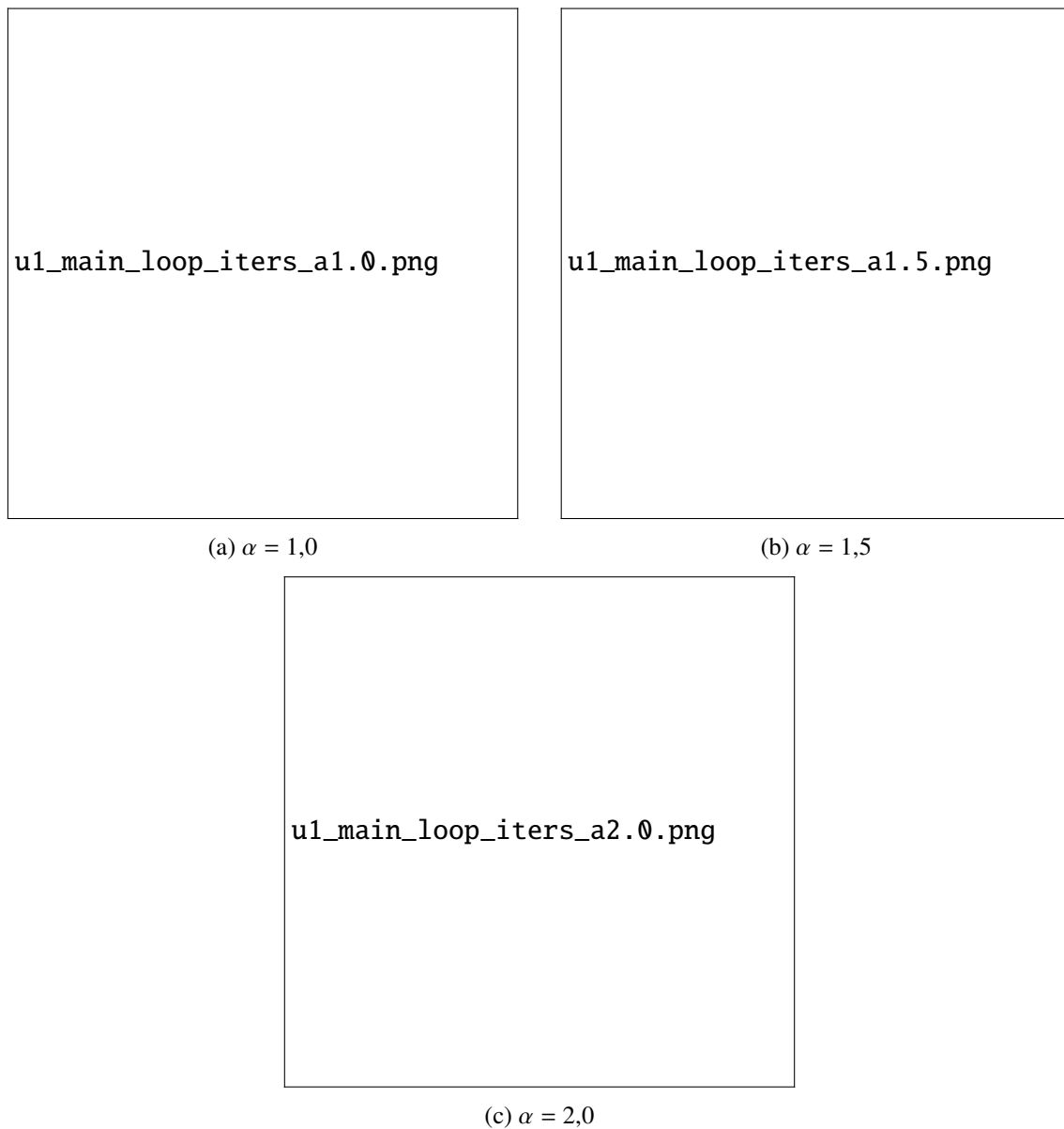


Figura 1.1 – Iterações do laço principal vs. n em escala log-log (U1). Simple cresce linearmente; HK mantém contagem constante para $\alpha \geq 1,5$ e cresce muito lentamente para $\alpha = 1,0$.

1.4.3 U2 – Custo da Busca Interna vs. Defeito

O teste U2 traça o custo da busca (arestas BFS visitadas) em função do *defeito* (vértices sem par) ao longo de uma única execução, mostrando como o trabalho interno evolui à medida que o emparelhamento cresce (Figura ??).

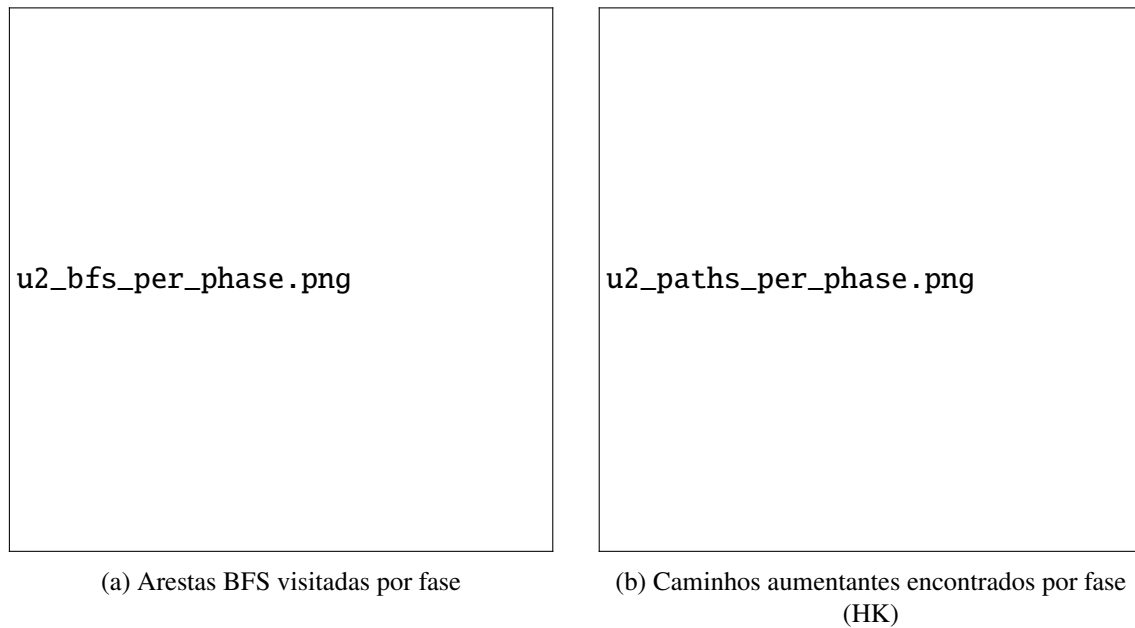


Figura 1.2 – Decomposição do trabalho interno por fase em U2. À esquerda: custo BFS por passo para Simple (crescente) vs. HK (limitado por m). À direita: número de caminhos vertex-disjoint encontrados por fase do HK, confirmando que cada fase aumenta múltiplos pares de forma simultânea.

Para o Simple, o custo por busca cresce monotonicamente conforme o defeito se aproxima de zero: as últimas aumentações precisam navegar caminhos alternantes mais longos, pois o emparelhamento atual bloqueia os atalhos. Para o HK, o custo por fase permanece em $O(m)$ independentemente do defeito, pois a BFS de construção de camadas visita cada aresta no máximo uma vez por fase. A Figura ??(b) confirma o mecanismo de eficiência do HK: cada fase encontra múltiplos caminhos vertex-disjoint simultâneos, amortizando o custo da BFS.

1.4.4 U3 – Escalabilidade em Tempo de Parede: Simple vs. HK

O teste U3 mede o tempo de parede em função de n para cinco valores de α , comparando diretamente Simple e HK em escala log-log.

Os resultados de U3 (Figura ??) revelam um padrão que contraria a intuição para



Figura 1.3 – Tempo de parede (mediana) vs. n em escala log-log por α (U3). Banda sombreada: mediana a p90; linhas tracejadas: referências teóricas. HK é superior para $\alpha \geq 1,5$; para $\alpha = 1,0$, Simple supera HK em todas as escalas testadas.

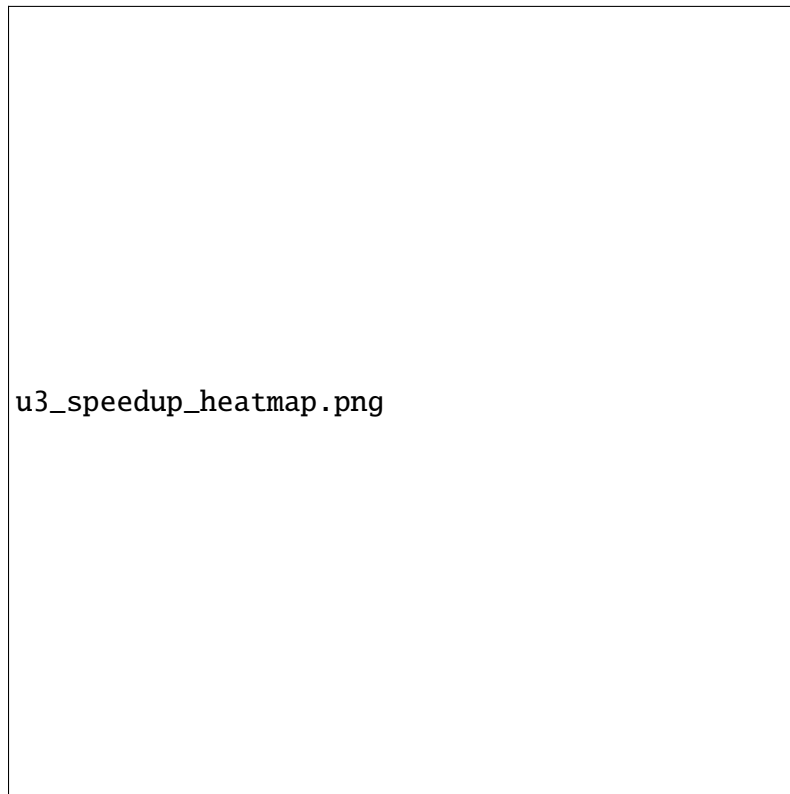


Figura 1.4 – Mapa de calor do fator de aceleração (Simple / HK) sobre (n, α) (U3). Valores < 1 (cores frias) indicam que Simple é mais rápido; valores > 1 (cores quentes) indicam HK mais rápido.

grafos esparsos: para $\alpha = 1,0$, o Simple é consistentemente **mais rápido** que HK em todos os tamanhos testados, com fator de aceleração $\approx 0,18$ (Simple é $\approx 5\times$ mais rápido), conforme evidenciado pelo mapa de calor da Figura ???. Para $\alpha = 1,5$, HK já é mais rápido a partir de $n \approx 200$. Para $\alpha = 2,0$, HK é dramaticamente superior.

A explicação para o caso esparsos está no *overhead* de HK: para grafos com $m = n$ arestas, cada BFS visita apenas $O(n)$ arestas e os caminhos aumentantes encontrados pelo Simple são curtos. O HK precisa inicializar o array `dist`, executar a BFS, e então fazer a DFS — todo esse overhead para encontrar o mesmo resultado que o Simple encontra com uma DFS simples. Nos grafos de $\alpha = 2,0$ (densos), HK termina em apenas **1 fase** para qualquer n , enquanto Simple precisa de n buscas, cada uma visitando $O(n^2)$ arestas.

Tabela 1.5 – Expoentes empíricos do tempo de parede: Simple vs. HK (U3).

α	Algoritmo	$k_{\text{teórico}}$	k_{emp}	R^2
1,0	SimpleAugmentingMatcher	2,00	1,96	1,000
1,0	HopcroftKarpMatcher	1,50	2,08	1,000
1,25	SimpleAugmentingMatcher	2,25	2,82	0,999
1,25	HopcroftKarpMatcher	1,75	1,90	1,000
1,5	SimpleAugmentingMatcher	2,50	3,01	1,000
1,5	HopcroftKarpMatcher	2,00	1,86	1,000
1,75	SimpleAugmentingMatcher	2,75	2,88	1,000
1,75	HopcroftKarpMatcher	2,25	1,85	1,000
2,0	SimpleAugmentingMatcher	3,00	2,99	1,000
2,0	HopcroftKarpMatcher	2,50	1,99	1,000

A Tabela ?? detalha os expoentes empíricos. Notavelmente, HK para $\alpha = 1,0$ apresenta $k_{\text{emp}} = 2,08 > k_{\text{teórico}} = 1,5$: o overhead de construção da rede em camadas inflaciona o expoente empírico para grafos esparsos. Para $\alpha = 2,0$, HK tem $k_{\text{emp}} = 1,99 \approx 2$, muito abaixo do teórico 2,5 — pois o número de fases é $O(1)$ em vez de $O(\sqrt{n})$, tornando a complexidade efetiva $O(m) = O(n^2)$ em vez de $O(\sqrt{n} \cdot m) = O(n^{2,5})$.

1.4.5 U4 – Sensibilidade à Densidade

Para um n fixo (Figura ??), o tempo do Simple cresce exponencialmente em α , pois cada busca visita $m = n^\alpha$ arestas, enquanto o HK responde de forma muito mais suave. A Figura ?? confirma que o número de fases do HK é praticamente independente de α para um n dado — em torno de 1–4 fases para todos os valores de α testados — o que é consistente com a teoria ($O(\sqrt{n})$ fases, sem dependência em m).

1.4.6 W1 – Convergência do Laço Principal: BF vs. JD

O teste W1 verifica que ambas as estratégias de caminho mais curto produzem trajetórias de déficit idênticas, confirmando que a escolha de BF vs. JD não altera o comportamento do laço externo do Húngaro.

Como mostra a Figura ??, em todas as instâncias testadas ($n \in \{50, 100, 200, 500, 1000\}$),

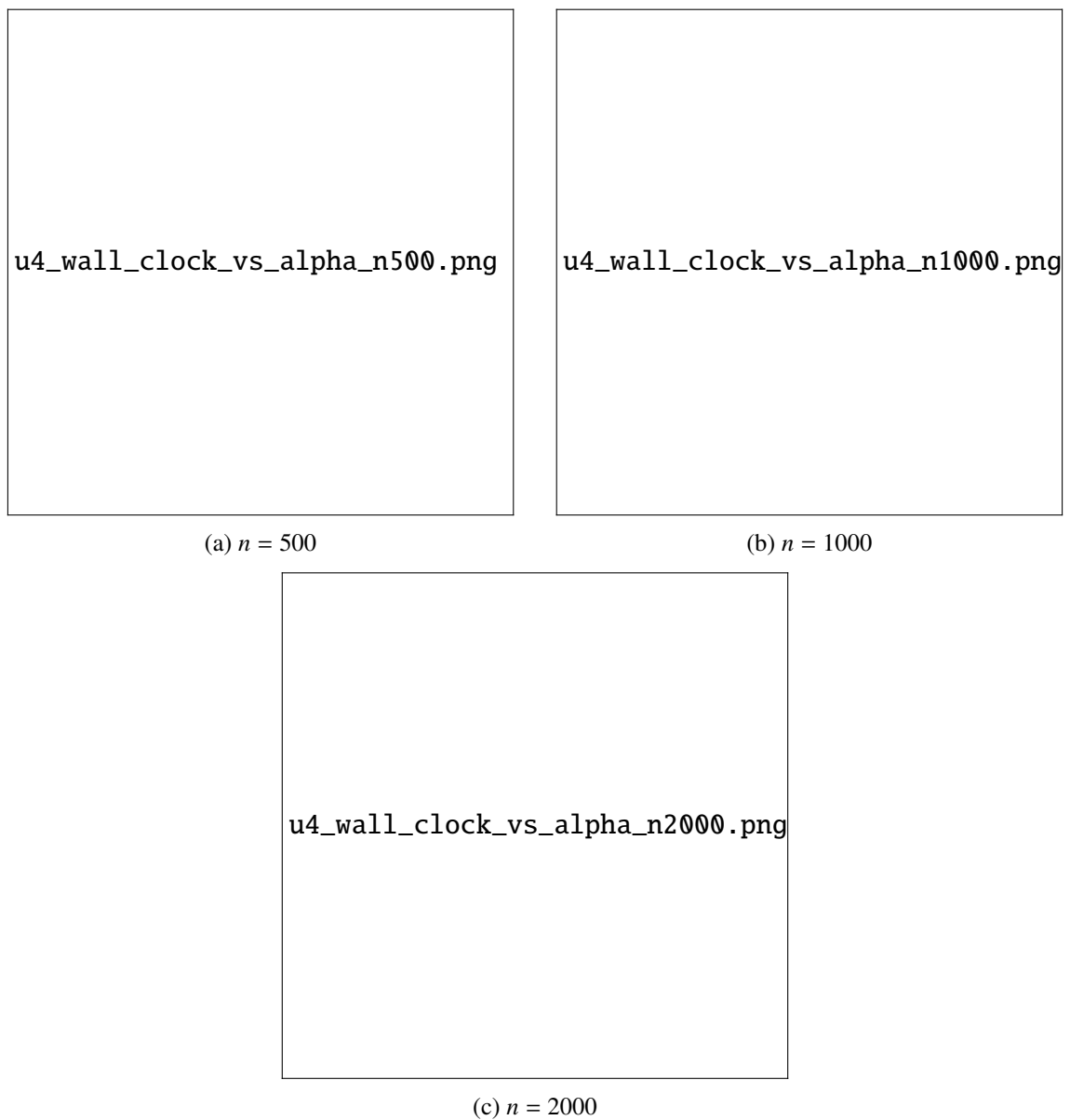


Figura 1.5 – Tempo de parede vs. α para n fixo (U4). Simple cresce exponencialmente em α ; HK é muito mais suave, com crescimento próximo a linear.



Figura 1.6 – Contagem de fases do HK vs. α para os três valores de n fixo (U4). As curvas são aproximadamente horizontais, confirmando que o número de fases de HK depende apenas de n e não de α .

$\alpha \in \{1,0; 1,5; 2,0\}$, 10 repetições), os valores de `augmentation_count` e `matching_value` foram idênticos para BF e JD, sem nenhuma divergência.

1.4.7 W2 – Custo por Chamada de Caminho Mais Curto

O teste W2 quantifica o custo interno por invocação do JD, medindo o número de relaxações de aresta em função de n para três valores de α .

Tabela 1.6 – Expoentes empíricos de relaxações de aresta por chamada (W2, JD).

α	Algoritmo	$k_{\text{teórico}}$	k_{emp}	R^2
1,0	JohnsonDijkstraStrategy	1,0	1,03	1,000
1,5	JohnsonDijkstraStrategy	1,5	1,50	1,000
2,0	JohnsonDijkstraStrategy	2,0	2,00	1,000

A Figura ?? e a Tabela ?? mostram que o número de relaxações de aresta por chamada do JD escala *perfeitamente* com n^α , confirmando a análise teórica de $O(m)$ relaxações por invocação do Dijkstra. O fator $\log n$ da fila de prioridade não aparece na contagem de relaxações pois cada aresta é relaxada no máximo uma vez; o custo do $\log n$

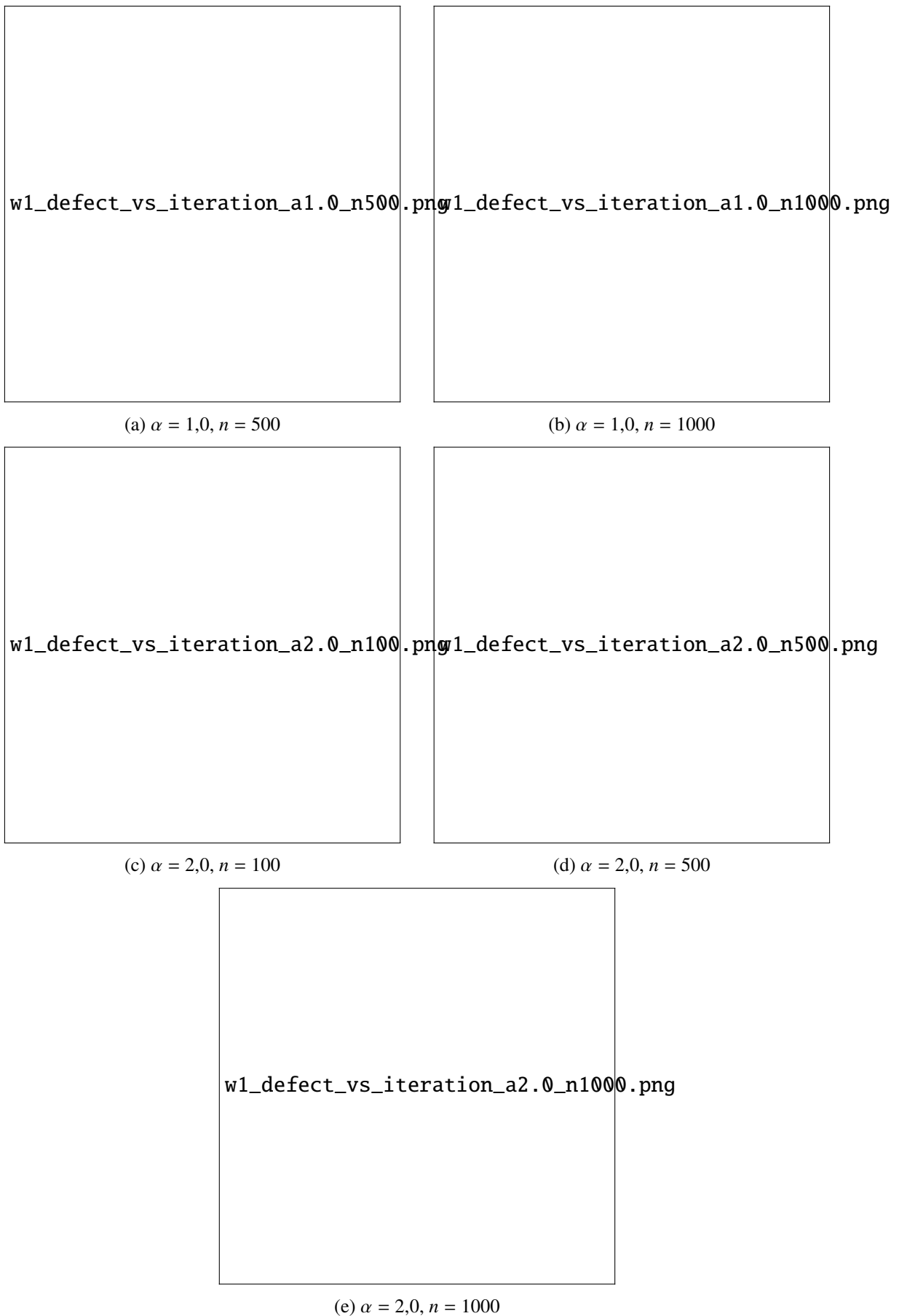


Figura 1.7 – Déficit vs. passo de aumentação para BF e JD (W1). As curvas de BF e JD se sobrepõem perfeitamente em todos os painéis, confirmando que ambas as estratégias executam exatamente as mesmas aumentações.

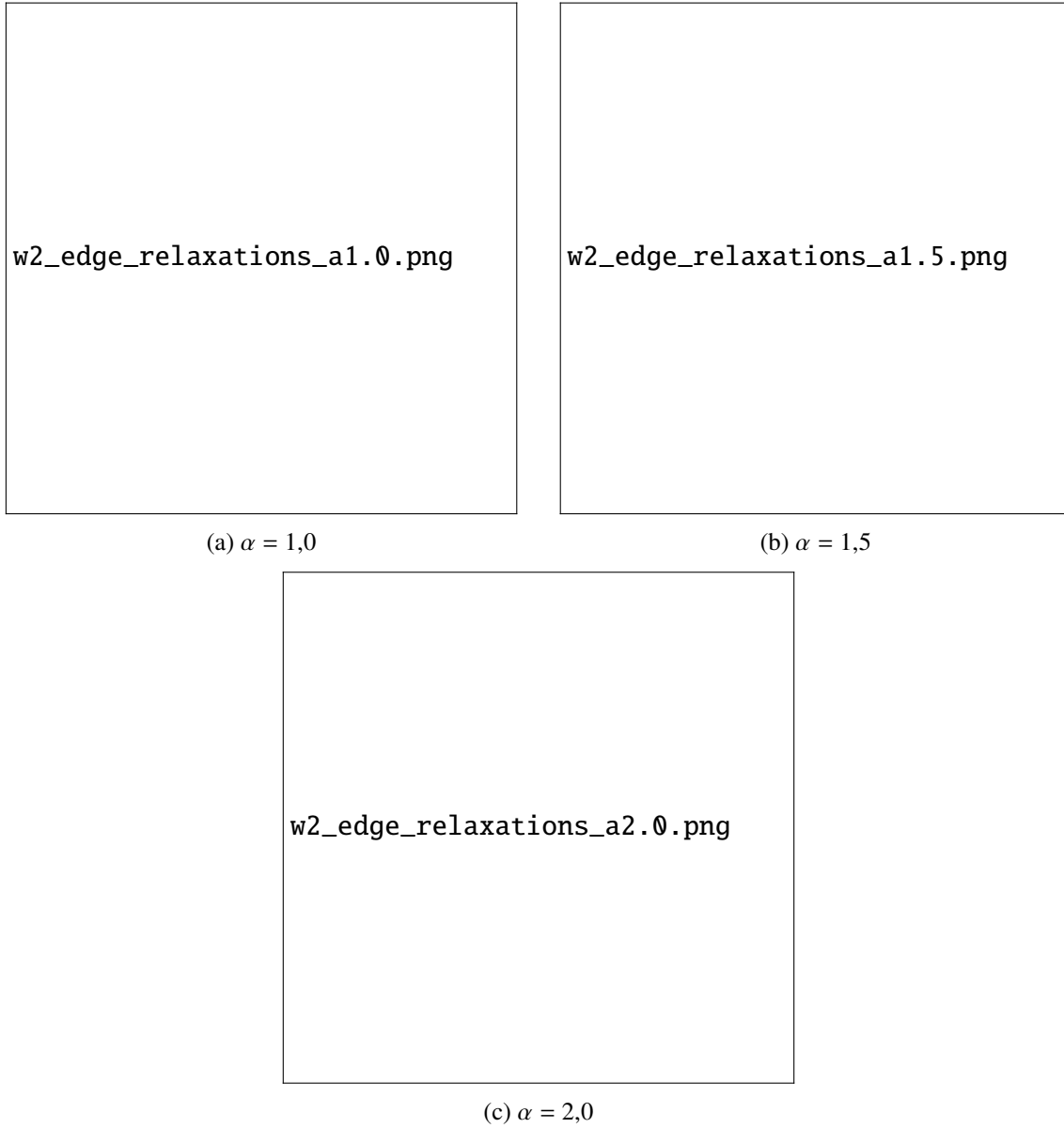


Figura 1.8 – Relaxações de aresta por chamada vs. n em escala log-log (W2, JD). Os expoentes empíricos coincidem com a teoria ($k_{\text{emp}} \approx \alpha$), confirmando $O(m)$ relaxações por invocação do Dijkstra.

está nas extrações da heap.

O contador `potential_violations_count` foi zero em **todas** as instâncias de W2, confirmando que a manutenção dos potenciais de Johnson garante $d'_{uv} \geq 0$ para toda aresta residual em todo momento.

1.4.8 W3 – Escalabilidade em Tempo de Parede: BF vs. JD

O teste W3 avalia o tempo de parede de BF e JD em função de n para cinco valores de α , repetindo o protocolo de U3 para os algoritmos ponderados.

Tabela 1.7 – Expoentes empíricos do tempo de parede: BF vs. JD (W3).

α	Algoritmo	$k_{\text{teórico}}$	k_{emp}	R^2
1,0	BellmanFordStrategy	3,00	3,02	1,000
1,0	JohnsonDijkstraStrategy	2,00	2,76	0,999
1,25	BellmanFordStrategy	3,25	3,09	1,000
1,25	JohnsonDijkstraStrategy	2,25	2,75	1,000
1,5	BellmanFordStrategy	3,50	2,87	1,000
1,5	JohnsonDijkstraStrategy	2,50	2,65	1,000
1,75	BellmanFordStrategy	3,75	2,91	1,000
1,75	JohnsonDijkstraStrategy	2,75	2,85	1,000
2,0	BellmanFordStrategy	4,00	2,97	1,000
2,0	JohnsonDijkstraStrategy	3,00	3,14	1,000

Os resultados de W3 (Figuras ?? e ??, Tabela ??) contêm o achado mais contraintuitivo dos experimentos: para $\alpha = 2,0$, JD é **10 vezes mais lento** que BF para $n = 2000$ (426 900 ms vs. 40 942 ms), apesar da previsão teórica de que JD deveria ser mais rápido. Dois efeitos concorrentes explicam o fenômeno:

1. **BF com terminação antecipada:** o expoente empírico de BF é $k_{\text{emp}} \approx 2,9\text{--}3,0$ para todos os α , *muito abaixo* do teórico $2 + \alpha = 4,0$ para $\alpha = 2,0$. A flag **relaxed** elimina os passos de relaxação desnecessários, e o acesso sequencial à matriz de pesos tem excelente localidade de cache.
2. **Overhead do heap de JD:** para $\alpha = 2,0$, há n^2 arestas. Cada chamada do Dijkstra pode inserir $O(n^2)$ entradas na fila de prioridade. As operações de heap têm alto custo constante e péssima localidade de cache comparadas às relaxações simples de

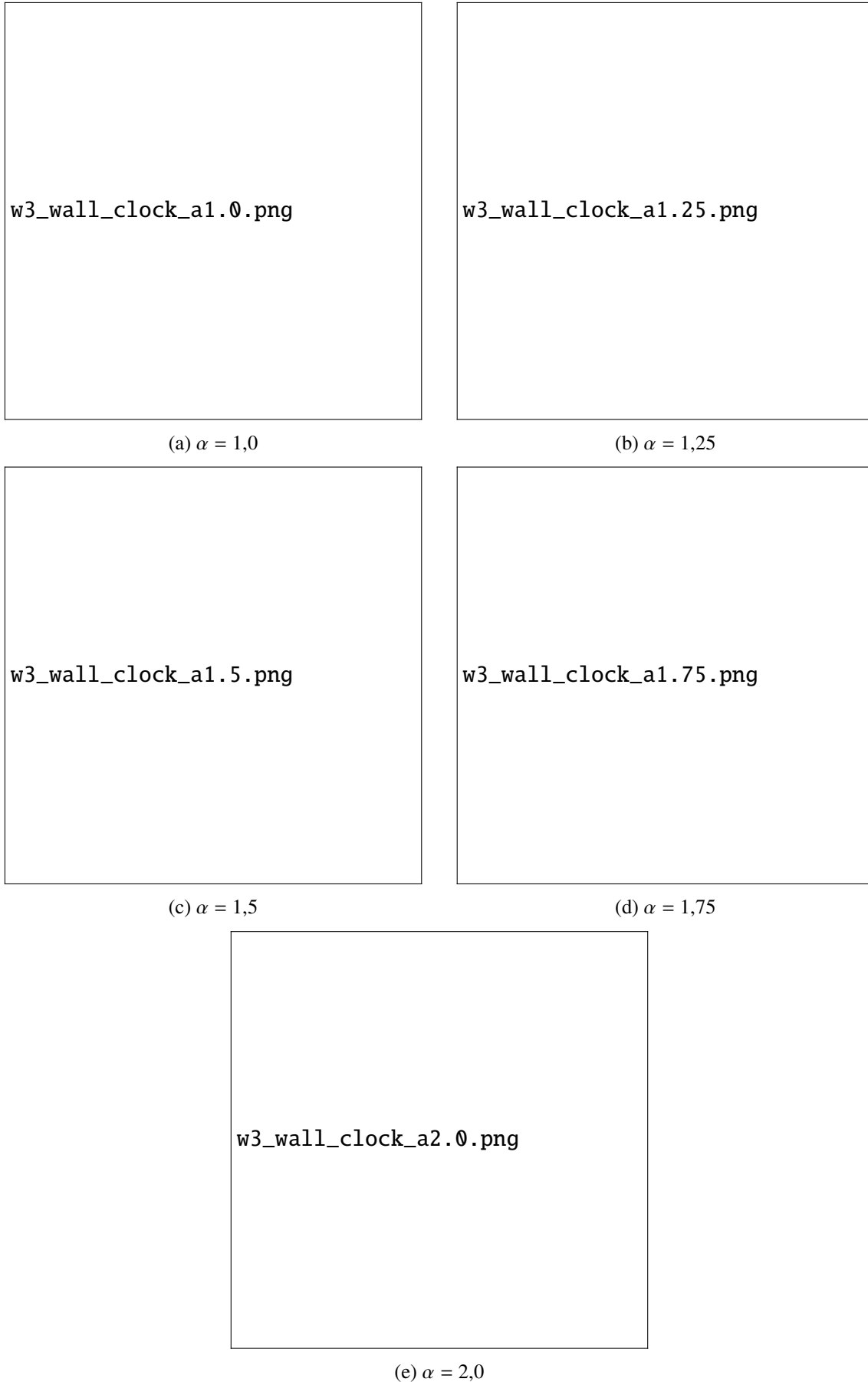


Figura 1.9 – Tempo de parede (mediana) vs. n em escala log-log por α (W3). Banda sombreada: mediana a p90; linhas tracejadas: referências teóricas. Para $\alpha \leq 1,5$, JD é mais rápido; para $\alpha = 2,0$, BF é substancialmente mais rápido.

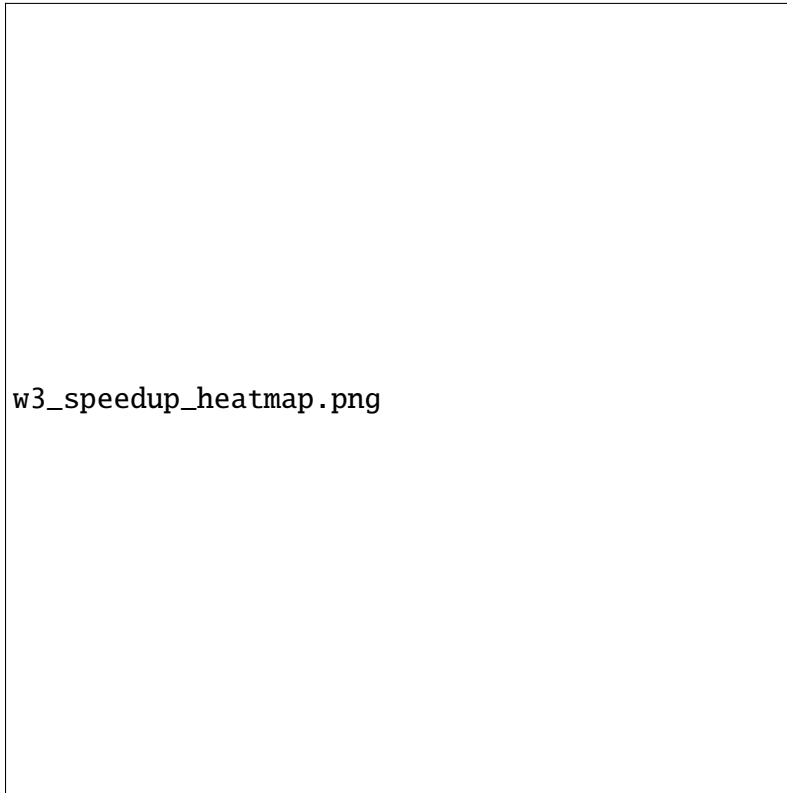


Figura 1.10 – Fator de aceleração (BF / JD) sobre (n, α) (W3). Para $\alpha = 2,0$, JD é consistentemente mais lento que BF (speedup < 1) em todo o intervalo de n .

BF. O expoente empírico de JD ($k_{\text{emp}} = 3,14$) supera o de BF ($k_{\text{emp}} = 2,97$) para $\alpha = 2,0$.

Para $\alpha = 1,0$, JD é $\sim 6\times$ mais rápido que BF a $n = 2000$, e para $\alpha = 1,5$, JD é $\sim 3\times$ mais rápido. A vantagem de JD é máxima em grafos esparsos, onde a fila de prioridade é pequena e o custo de BF por passo é dominado pelo percurso de todos os n vértices.

1.4.9 W4 – Estresse com Pesos Negativos

O teste W4 compara os três regimes de peso (Figura ??): misto $[-n^2, n^2]$, totalmente negativo $[-n^2, 0]$ e totalmente positivo $[0, n^2]$. Em **todas as combinações** de $(n, \alpha, \text{regime})$, BF e JD produziram valores de emparelhamento *idênticos* (diferença absoluta $< 10^{-6}$). O contador `potential_violations_count` permaneceu **zero em todos os regimes**, incluindo o totalmente negativo — onde o potencial inicial $p_v = -W$ é mais tensionado.

Um resultado curioso é que os valores de emparelhamento para o regime misto e o regime negativo foram **idênticos** para $n = 500$ e $n = 1000$. Isso indica que, para os grafos gerados, o emparelhamento perfeito de máxima cardinalidade no regime misto escolhe



Figura 1.11 – Tempo de parede por regime de peso ($\alpha = 1,5$) para BF e JD (W4). Os três regimes (misto, negativo, positivo) apresentam tempos similares dentro de cada algoritmo, confirmando que o regime de peso não afeta o custo de convergência.

automaticamente arestas com pesos negativos — a estrutura do grafo esparso não oferece alternativas de alto peso para todos os n vértices da partição.

1.4.10 C1 – Comparação Final: Algoritmos Não Ponderados

A Figura ?? e as Tabelas ?? e ?? apresentam os resultados finais de escalabilidade para os algoritmos não ponderados.

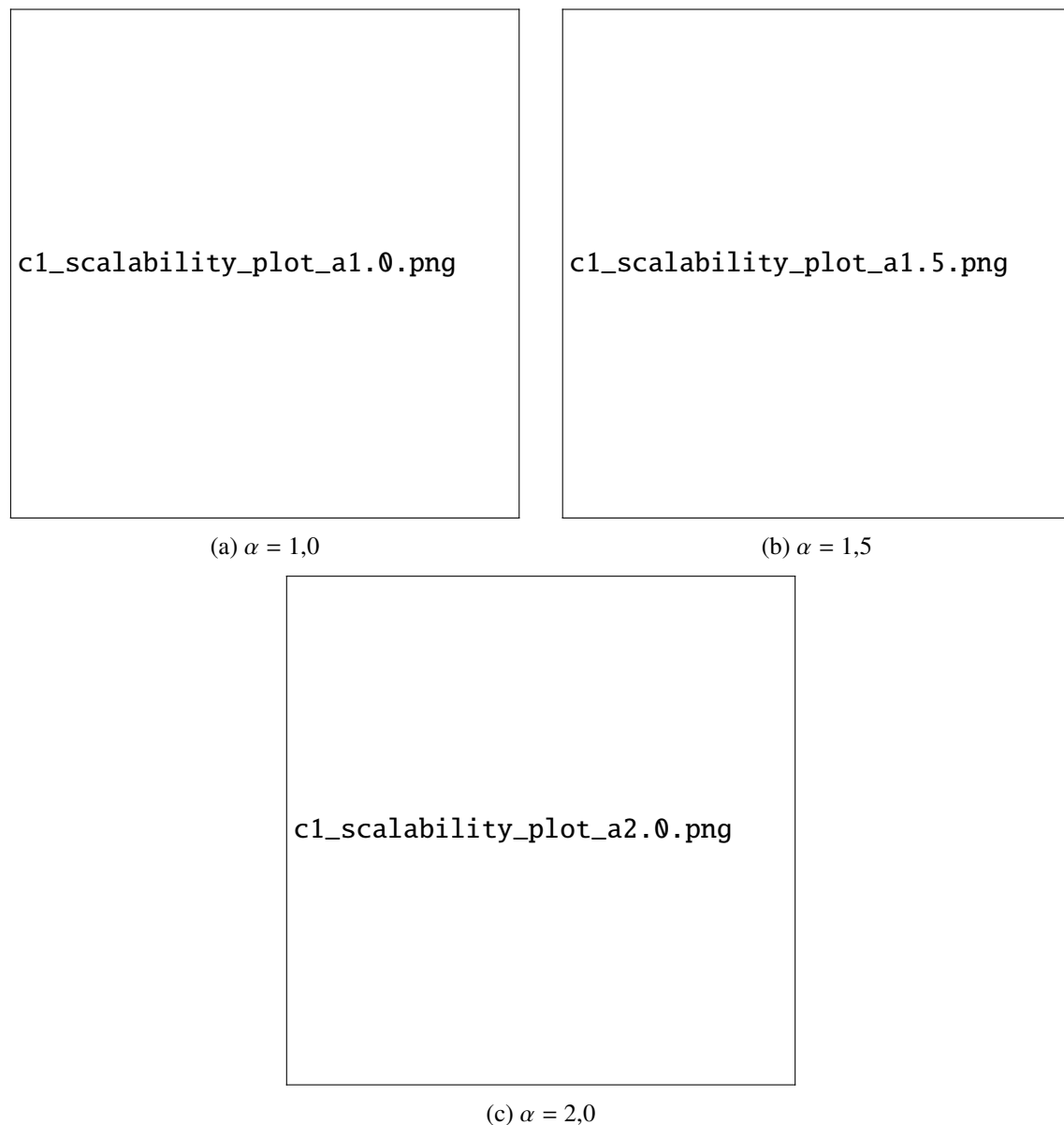


Figura 1.12 – Escalabilidade final: Simple vs. HK (C1, 30 repetições). Bandas de variação mediana–p90 sobrepostas. HK domina para $\alpha \geq 1,5$; Simple é mais eficiente para $\alpha = 1,0$.

Tabela 1.8 – Tempos médios (ms) e fator de aceleração (C1, 30 repetições). Speedup = Simple/HK; valores < 1 indicam que Simple é mais rápido.

$\alpha = 1,0$				$\alpha = 1,5$				$\alpha = 2,0$			
n	Simple	HK	Speedup	n	Simple	HK	Speedup	n	Simple	HK	Speedup
500	0,4	1,7	0,21	500	25,4	1,1	23,7	500	35,4	0,5	67,7
1000	1,4	7,3	0,19	1000	151,2	2,8	54,4	1000	219,5	2,0	109,3
2000	4,0	22,7	0,17	2000	1167	9,9	118,2	2000	1729,7	8,0	216,6
5000	23,3	127,4	0,18	5000	18602	56,9	327,2	5000	26835	49,7	539,9
10000	93,3	572,9	0,16	10000	151948	218,2	696,5	10000	215196	198,7	1083,2

Tabela 1.9 – Expoentes empíricos ajustados (C1).

α	Algoritmo	$k_{\text{teórico}}$	k_{emp}	Δk	R^2
1,0	SimpleAugmentingMatcher	2,00	1,83	-0,17	0,998
1,0	HopcroftKarpMatcher	1,50	1,91	+0,41	0,998
1,5	SimpleAugmentingMatcher	2,50	2,92	+0,42	0,999
1,5	HopcroftKarpMatcher	2,00	1,80	-0,20	0,997
2,0	SimpleAugmentingMatcher	3,00	2,93	-0,07	1,000
2,0	HopcroftKarpMatcher	2,50	1,99	-0,51	1,000

Os desvios em k para HK em $\alpha = 1,0$ (+0,41) e $\alpha = 2,0$ (-0,51) refletem os dois regimes distintos: para $\alpha = 1,0$, HK paga overhead extra de inicialização por fase; para $\alpha = 2,0$, HK opera em $O(1)$ fases e sua complexidade efetiva é $O(m) = O(n^2)$, não $O(n^{2,5})$.

O fator de aceleração de HK sobre Simple cresce aproximadamente como $n^{0,7}$ para $\alpha = 1,5$ (de $23\times$ em $n = 500$ a $697\times$ em $n = 10000$) e como $n^{0,9}$ para $\alpha = 2,0$ (de $68\times$ em $n = 500$ a $1083\times$ em $n = 10000$).

1.4.11 C2 – Comparação Final: Algoritmos Ponderados

A Figura ?? e as Tabelas ?? e ?? apresentam os resultados finais de escalabilidade para os algoritmos ponderados.

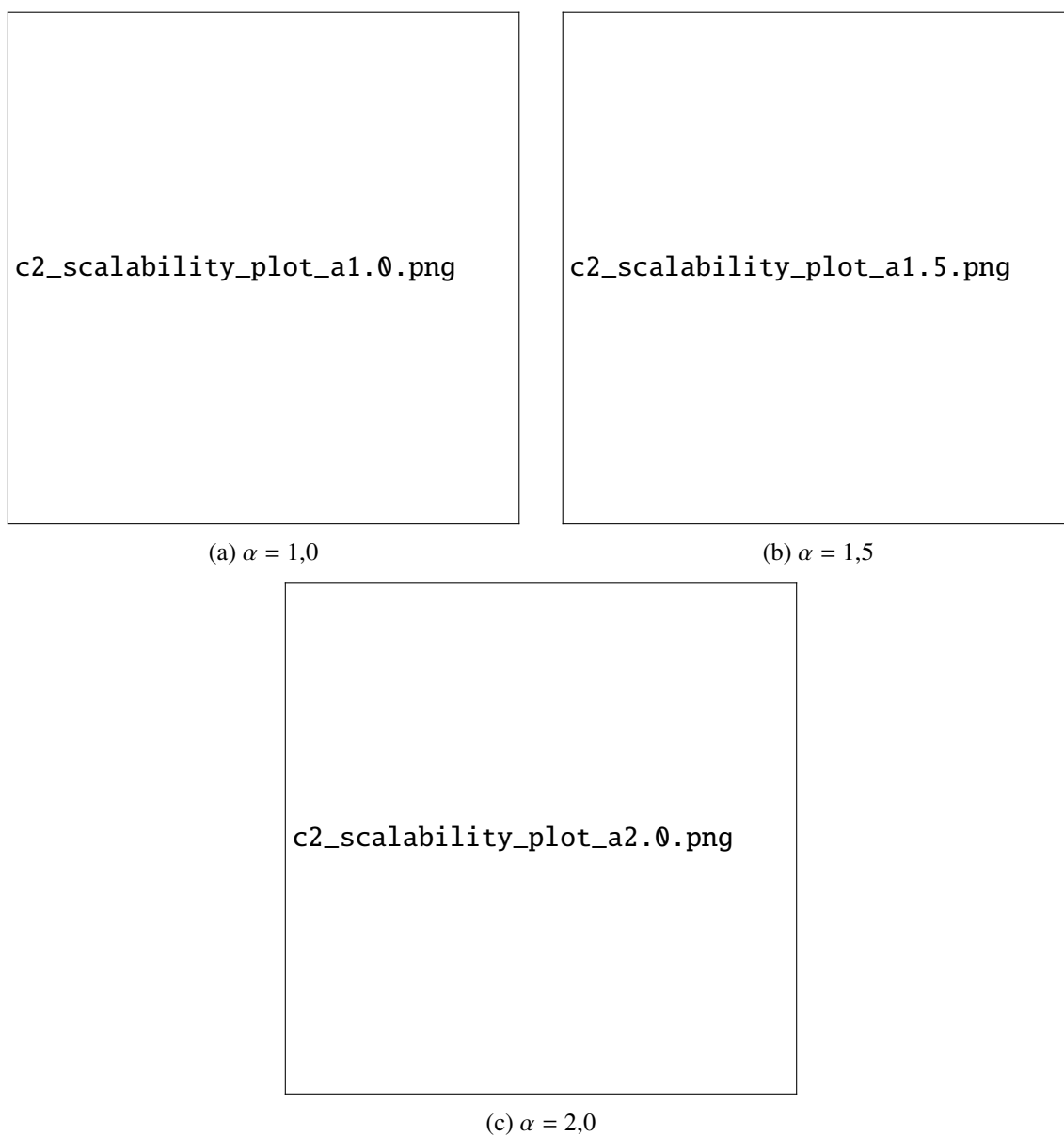


Figura 1.13 – Escalabilidade final: BF vs. JD (C2, 20 repetições). JD domina para $\alpha \leq 1,5$; BF é mais rápido para $\alpha = 2,0$.

Tabela 1.10 – Tempos médios (ms) e fator de aceleração BF/JD (C2, 20 repetições). Speedup < 1 indica JD mais lento que BF.

$\alpha = 1,0$				$\alpha = 1,5$				$\alpha = 2,0$			
n	BF	JD	Speedup	n	BF	JD	Speedup	n	BF	JD	Speedup
100	2,4	0,9	2,57	100	9,9	6,3	1,55	100	5,8	34,2	0,17
200	19,2	5,3	3,65	200	75,2	36,8	2,04	200	46,7	288,7	0,16
500	311,9	67,8	4,60	500	948,5	397,7	2,38	500	640,8	5150	0,12
1000	2602,2	486,6	5,35	1000	6779,2	2571,1	2,64	1000	5099	46254	0,11
2000	22271	3720,2	5,99	2000	51286	17972	2,85	2000	40942	426901	0,10

Tabela 1.11 – Expoentes empíricos ajustados (C2).

α	Algoritmo	$k_{\text{teórico}}$	k_{emp}	Δk	R^2
1,0	BellmanFordStrategy	3,00	3,05	+0,05	1,000
1,0	JohnsonDijkstraStrategy	2,00	2,78	+0,78	0,999
1,5	BellmanFordStrategy	3,50	2,84	-0,66	1,000
1,5	JohnsonDijkstraStrategy	2,50	2,65	+0,15	1,000
2,0	BellmanFordStrategy	4,00	2,95	-1,05	1,000
2,0	JohnsonDijkstraStrategy	3,00	3,15	+0,15	1,000

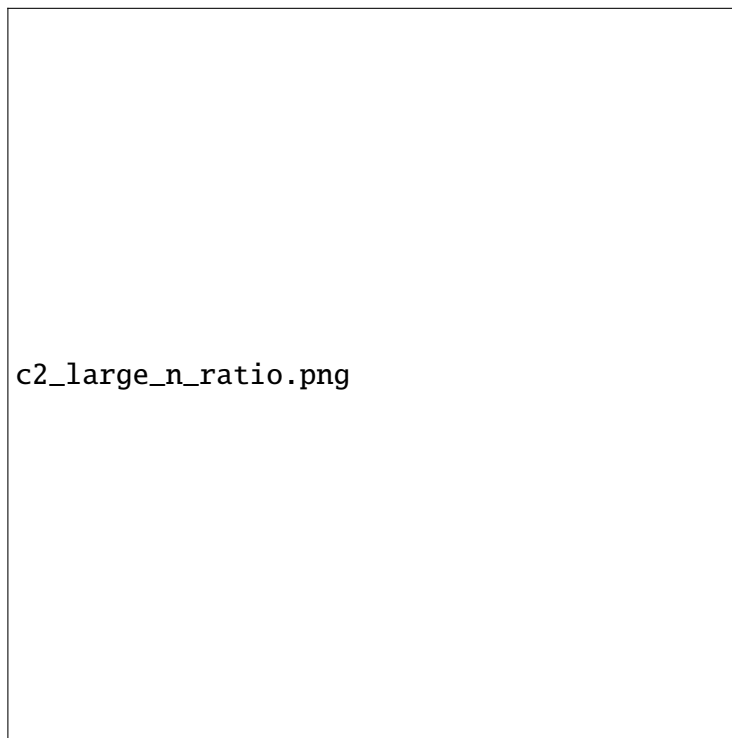


Figura 1.14 – Speedup empírico BF/JD vs. previsão teórica $n/\log_2 n$ para $\alpha = 2,0$ (C2). A razão empírica está abaixo de 1 em toda a faixa, contradizendo a previsão de que JD deveria ser mais rápido para grafos densos.

O desvio mais expressivo ocorre para BF em $\alpha = 2,0$: $k_{\text{emp}} = 2,95$ vs. $k_{\text{teórico}} = 4,00$ ($\Delta k = -1,05$). A terminação antecipada do Bellman-Ford reduz os passos efetivos de $2n$ para um número muito menor na prática. A Figura ?? mostra que o speedup empírico para $\alpha = 2,0$ é inferior a 1 (BF é mais rápido), ao contrário da previsão teórica de $n/\log_2 n$.

1.5 Conclusão

Os experimentos confirmaram as complexidades teóricas para a maioria dos casos, mas revelaram nuances importantes:

Algoritmos não ponderados: O HK é superior ao Simple para $\alpha \geq 1,5$, com ganhos que crescem rapidamente com n (de $\sim 24\times$ a $\sim 1083\times$ entre $n = 500$ e $n = 10000$ para $\alpha = 2,0$). Contudo, para $\alpha = 1,0$ (grafos esparsos), Simple é $\sim 5\times$ mais rápido em toda a faixa testada: o overhead de inicialização de fase do HK supera o custo das buscas simples quando os caminhos são curtos e a estrutura em camadas oferece pouco ganho.

Algoritmos ponderados: JD é claramente superior para $\alpha = 1,0$ ($\sim 6\times$ mais rápido a $n = 2000$) e moderadamente melhor para $\alpha = 1,5$ ($\sim 3\times$). O resultado surpreendente é que para $\alpha = 2,0$, BF supera JD por um fator de $\sim 10\times$: a terminação antecipada reduz o expoente efetivo de BF para $\approx 3,0$ (muito abaixo do teórico $4,0$), enquanto o alto custo constante do heap de JD em grafos densos resulta em expoente empírico de $3,15$ — marginalmente acima do de BF. Esse cruzamento é um exemplo típico de como fatores de implementação (localidade de cache, overhead de estruturas de dados, terminação antecipada) podem reverter previsões assintóticas em faixas práticas de n .

Implicação prática: Para emparelhamentos não ponderados, recomenda-se HK para qualquer grafo com $\alpha \geq 1,5$; para $\alpha = 1,0$, Simple é preferível até $n = 10000$. Para emparelhamentos ponderados com grafos esparsos a médios ($\alpha \leq 1,5$), JD é a escolha correta. Para grafos densos ($\alpha \approx 2,0$), BF com terminação antecipada apresenta desempenho inesperadamente competitivo, e JD só deve ser preferido se o n for suficientemente grande para amortizar o overhead da heap.

APÊNDICE A — TABELAS COMPLEMENTARES DE OPERAÇÕES

A.1 Contagem de Operações: Algoritmos Não Ponderados (C1)

Tabela A.1 – Médias de fases e arestas visitadas por HK (C1, $\text{iteration} = -1$).

n	α	Fases médias (HK)	Arestas visitadas médias (HK)
500	1,0	2,50	1 328
500	1,5	2,00	24 394
500	2,0	1,00	375 250
1000	1,0	2,93	2 889
1000	1,5	2,00	67 886
1000	2,0	1,00	1 500 500
2000	1,0	3,27	6 151
2000	1,5	2,00	186 394
2000	2,0	1,00	6 001 000
5000	1,0	3,57	16 040
5000	1,5	2,00	731 347
5000	2,0	1,00	37 502 500
10000	1,0	4,13	34 981
10000	1,5	2,00	2 054 085
10000	2,0	1,00	150 005 000

Para $\alpha = 2,0$, HK executa exatamente 1 fase e visita aproximadamente $n^2/2$ arestas na BFS, confirmando que uma única passagem percorre todo o grafo. Para $\alpha = 1,5$, HK usa consistentemente 2 fases independentemente de n , enquanto para $\alpha = 1,0$ cresce logaritmicamente (de 2,5 a 4,1 fases).